

WEEK-4 : CUSTOMER AND PRODUCT ORDER ANALYSIS.

-S.KAVIYA(E25AI119)

Scenario:

Art Work Shop manages customer orders and product sales using a database system. The database stores customer, product and order details. The reports help to view customer order history, product-wise order quantities, and customer purchase details. These reports make it easier to analyse the workshop's orders and customer purchases.

Create Table – Orders:

The Orders table is created to store the details of customer orders. It contains the order ID, customer ID, order date, total amount and order status. Primary key, foreign key, NOT NULL and CHECK constraints are included while creating the table.

```
SQL> CREATE TABLE Orders (Order_ID NUMBER PRIMARY KEY, Customer_ID VARCHAR2(20) NOT NULL, Order_Date DATE NOT NULL, Total_Amount NUMBER(10,2) NOT NULL, Order_Status VARCHAR2(20) NOT NULL, CONSTRAINT fk_orders_customer FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID), CONSTRAINT chk_order_status CHECK (Order_Status IN ('Pending','Shipped','Delivered')));  
Table created.
```

Create Table – Order_Details:

The Order_Details table is created to store the details of products included in each order. It contains the order detail ID, order ID, product ID, quantity and price. Primary key, foreign key, NOT NULL and CHECK constraints are included while creating the table.

```
SQL> CREATE TABLE Order_Details (Order_Detail_ID NUMBER PRIMARY KEY, Order_ID NUMBER NOT NULL, Product_ID VARCHAR2(7) NOT NULL, Quantity NUMBER NOT NULL, Price NUMBER(10,2) NOT NULL, CONSTRAINT fk_od_order FOREIGN KEY(Order_ID) REFERENCES Orders(Order_ID), CONSTRAINT fk_od_product FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID), CONSTRAINT chk_od_quantity CHECK(Quantity > 0), CONSTRAINT chk_od_price CHECK(Price > 0));  
Table created.  
SQL>
```

Establish Relationship Using Primary Key and Foreign Key:

The relationship between the tables is established using Primary Key (PK) and Foreign Key (FK).

The CUSTOMER_ID in Customer --- CUSTOMER_ID in Orders.

The ORDER_ID in Orders ---ORDER_ID in Order_Details.

The PRODUCT_ID in Product ---- PRODUCT_ID in Order_Details.

These relationships connect the customer, order and product information in the database.

Apply Constraints:

The database tables are created with different constraints to maintain data accuracy and integrity.

Primary Key- It is used to uniquely identify each record.

Foreign Key-It is used to establish relationships between the tables.

NOT NULL- It ensures that required fields cannot be left empty.

CHECK constraints- It ensure that only valid values are entered into the tables.

Inserting Values:

```
SQL> INSERT ALL
  2   INTO Order_Details VALUES (1, 1001, 'P1001', 1, 67500)
  3   INTO Order_Details VALUES (2, 1002, 'P1002', 1, 45000)
  4   INTO Order_Details VALUES (3, 1003, 'P1003', 1, 3400)
  5   INTO Order_Details VALUES (4, 1004, 'P1004', 1, 75000)
  6   INTO Order_Details VALUES (5, 1005, 'P1005', 1, 7500)
```

```
SQL> INSERT ALL
  2   INTO Orders VALUES (1001, 'C-1001', TO_DATE('2026-09-20', 'YYYY-MM-DD'), 67500, 'Pending')
  3   INTO Orders VALUES (1002, 'C-1002', TO_DATE('2026-09-21', 'YYYY-MM-DD'), 45000, 'Shipped')
  4   INTO Orders VALUES (1003, 'C-1003', TO_DATE('2026-09-22', 'YYYY-MM-DD'), 3400, 'Delivered')
  5   INTO Orders VALUES (1004, 'C-1004', TO_DATE('2026-09-23', 'YYYY-MM-DD'), 75000, 'Pending')
  6   INTO Orders VALUES (1005, 'C-1005', TO_DATE('2026-09-24', 'YYYY-MM-DD'), 7500, 'Shipped')
```

```
SQL> SELECT * FROM Orders
2 ;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
1001	C-1001	20-SEP-26	67500	Pending
1002	C-1002	21-SEP-26	45000	Shipped
1003	C-1003	22-SEP-26	3400	Delivered
1004	C-1004	23-SEP-26	75000	Pending
1005	C-1005	24-SEP-26	7500	Shipped

```
SQL> SELECT * FROM Order_Details;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT	QUANTITY	PRICE
1	1001	P1001	1	67500
2	1002	P1002	1	45000
3	1003	P1003	1	3400
4	1004	P1004	1	75000
5	1005	P1005	1	7500

Update Record

The existing order record is updated to modify the total amount.

```
SQL> UPDATE Orders
2 SET TOTAL_AMOUNT = 70000
3 WHERE ORDER_ID = 1001;

1 row updated.
```

Delete Record

An existing order record is deleted from the Orders table

```

SQL> DELETE FROM Order_Details
  2  WHERE ORDER_ID = 1005;

1 row deleted.

SQL> DELETE FROM Orders
  2  WHERE ORDER_ID = 1005;

1 row deleted.

SQL>

```

Report 1 – Customer Order History

This report displays the order history of each customer, including their name, order ID, order date, total amount and order status.

```

SQL> SELECT FIRST_NAME, LAST_NAME, ORDER_ID, ORDER_DATE, TOTAL_AMOUNT
  2  FROM Customer
  3  JOIN Orders
  4  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID;

```

FIRST_NAME	LAST_NAME	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT
Beema	ponting	1001	20-SEP-26	67500
priya	sakthi	1002	21-SEP-26	45000
Arun	kumar	1003	22-SEP-26	3400
Anu	Priya	1004	23-SEP-26	75000
Reena	das	1005	24-SEP-26	7500

Report 2 – Product-wise Order Report

This report displays the total quantity ordered for each product. It helps to analyse the products based on their order quantities.

3/2 0734: unknown command beginning 'ON Product...' - rest of line ignored.

```
SQL>
SQL> SELECT Product_Name,
2      SUM(Quantity) AS Total_Quantity
3 FROM Product
4 JOIN Order_Details
5 ON Product.Product_ID = Order_Details.Product_ID
6 GROUP BY Product.Product_ID, Product.Product_Name;
```

```
PRODUCT_NAME
-----
TOTAL_QUANTITY
-----
Laptops
      1

Unruled notes
      1

Croptops and shirts
      1

PRODUCT_NAME
-----
TOTAL_QUANTITY
-----
Fish tank
      1

Television
      1
```

Report 3 – Customer Purchase Analysis

This report analyses customer purchases by finding the customers with maximum orders, total spending of each customer, and their average order value.

❖ Customers with Maximum Orders

Identifies the customers who have placed the maximum number of orders.

```
SQL> SELECT Customer.FIRST_NAME, Customer.LAST_NAME, COUNT(Orders.ORDER_ID) AS TOTAL_ORDERS FROM Customer JOIN Orders ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID GROUP BY Customer.CUSTOMER_ID, Customer.FIRST_NAME, Customer.LAST_NAME HAVING COUNT(Orders.ORDER_ID) = (SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(ORDER_ID) AS ORDER_COUNT FROM Orders GROUP BY CUSTOMER_ID));
```

FIRST_NAME	LAST_NAME	TOTAL_ORDERS
priya	sakthi	2
Arun	kumar	2
Anu	Priya	2
Beema	ponting	2

❖ Total Spending by Each Customer

Calculates the total amount spent by each customer.

```
SQL> SELECT Customer.FIRST_NAME, Customer.LAST_NAME, SUM(Orders.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Customer JOIN Orders ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID GROUP BY Customer.CUSTOMER_ID, Customer.FIRST_NAME, Customer.LAST_NAME;
```

FIRST_NAME	LAST_NAME	TOTAL_SPENDING
priya	sakthi	112500
Arun	kumar	48400
Anu	Priya	78400
Reena	das	7500
Beema	ponting	125000

❖ Average Order Value

Calculates the average order value for each customer.

```
SQL> SELECT Customer.FIRST_NAME, Customer.LAST_NAME, AVG(Orders.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM Customer JOIN Orders ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID GROUP BY Customer.CUSTOMER_ID, Customer.FIRST_NAME, Customer.LAST_NAME;
```

FIRST_NAME	LAST_NAME	AVERAGE_ORDER_VALUE
priya	sakthi	56250
Arun	kumar	24200
Anu	Priya	39200
Reena	das	7500
Beema	ponting	62500